POLYGLOT WORKSHOP

POST-TRAINING AND ALIGNMENT

Tutors

Florian Mai & Nicholas Kluge

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

AGENDA

1. Post-training

- ✓ Why → What → How
- ✓ Evals
- ✓ Frameworks
- ✓ Chat Templates

2. Post-training Datasets

- ✓ Conditional Generation & Dataset Types
- ✓ Dataset Creation
- ✓ Constitutional AI

3. Supervised Fine-Tuning

- ✓ SFT Knobs and Dials

3. Preference Optimization

- ✓ PO Knobs and Dials

POST-TRAINING

Why → What → How

POST-TRAINING / WHY

Pre-training gives us a base model with **raw ability**. Post-training is where we shape that capability into something **reliable** and **steerable**. If pretraining is about **brute-forcing knowledge** into weights, post-training is about **sculpting that raw capability** into something reliable and steerable. But before setting the GPUs on fire again, ask yourself, ***should we even do post-training?***

Why post-train?

- ✔ **Research**: Exploring whether RL can unlock new reasoning capabilities in an existing model.
- ✔ **Production**: Distill a large model into a smaller one for latency reasons.
- ✔ **Gap**: No strong open model exists for a specific use case.
- ✘ Because everyone does it!

POST-TRAINING / WHAT

Post-training builds on existing capabilities. However, for this, you need the appropriate ***"type of sauce"*** to elicit them from the model. Unlike pretraining, **you don't need trillions of tokens**; you only need a **handful** of well-crafted samples (LIMA: Less Is More for Alignment).

What should post-training achieve?

- ✔ Do you want a crisp instruction-follower?
- ✔ A versatile assistant that can be easily steered?
- ✔ A reasoning engine for agentic problems?
- ✔ A model that can converse in a specific language?

If the awenser the following questions is "NO", maybe don't post-train

- ✘ Do you have access to high-quality, domain-specific data?
- ✘ Can you measure success?

POST-TRAINING / HOW

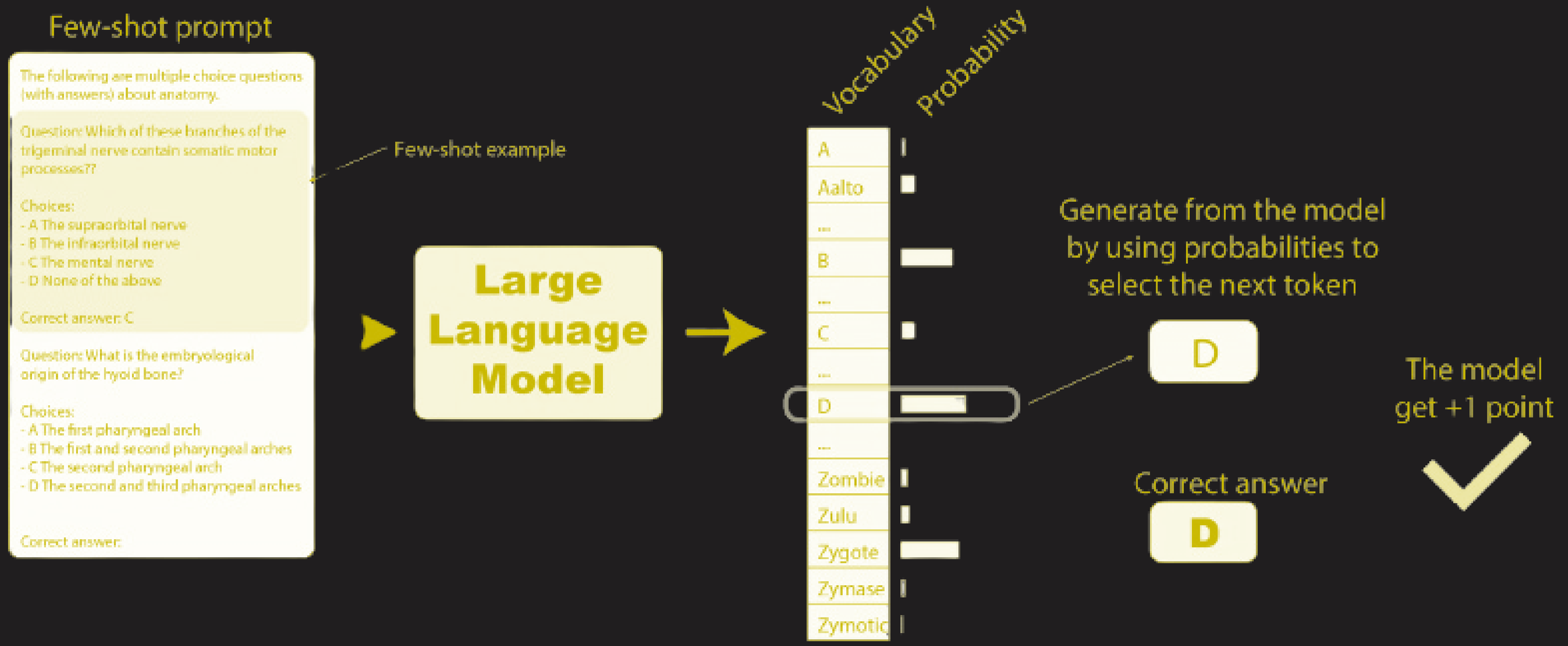
Since the early days of InstructGPT (March, 2022), the alignment and post-training recipe has evolved into a multi-stage pipeline that is now nearly universal across model development.

How will you get there?

- ✔ **Data curation** → to strike the right balance between diversity and quality.
- ✔ **Evaluation** → to track progress and catch regressions early.
- ✔ **Supervised fine-tuning** → to instill core capabilities (i.e., Imitation learning).
- ✔ **Preference Optimization** → to directly learn from preferences (i.e., preference learning).
- ✔ ~~**Reinforcement learning (RL)** → to refine reliability and reasoning beyond supervised data.~~

EVALS IN POST-TRAINING

- ✓ **For post-training, we prioritize using generative evaluations.**
- ✓ **Log-likelihood with a MQA format is still used, but some consider bad practice.**



Evals always should come before everything else.

[Source → The LLM Evaluation Guidebook]

EVALS IN POST-TRAINING

Instruction Following

>> 🏆 **IFEval (🔗)** → Follow formatting instructions. Solutions are verified using a dedicated parsing test.

Mathematics

>> 🏆 **GSM8K (🔗)** → Grade school (primary school) math problems.

Coding

>> 🏆 **HumanEval (🔗)** → Code completion.

REMEMBER …

- ✓ Always include a small set of **vibe evals**.
- ✓ Use **small subsets to accelerate evals** during model development.
- ✓ For **reasoning models** , strip the chain-of-thought, and give them space to reason.
- ✓ If an eval uses **LLM judges, pin the judge and version** for apples-to-apples comparisons.
- ✓ Keep a set of held-out benchmarks for the final model reports.
- ✓ When implementing a new eval, make sure you **can replicate published results.**
- ✓ When in doubt, **inspect the samples**.

TOOLS AND FRAMEWORKS

Framework	SFT	PO	RL	Multi-Modal	FullFT	LoRA	Distributed
TRL	✓	✓	✓	✓	✓	✓	✓
Axolotl	✓	✓	✓	✓	✓	✓	✓
Unsloth	✓	✓	✓	✓	✓	✓	✓
OpenInstruct	✓	✓	✓	✗	✓	✓	✓
vERL	✓	✗	✓	✓	✓	✓	✓
OpenRLHF	✓	✓	✓	✗	✓	✓	✓
NemoRL	✓	✓	✓	✗	✓	✗	✓

***Use frameworks; reinvent selectively**!

CHAT TEMPLATES

- ✓ **Chat templates** can be understood as the structure that bounds the user-model interaction. Much like a grammatical framework enables language to be meaningful, chat templates provide the formal organization that allows a language model to interpret roles, context, and intent.

- ✓ At their core, chat templates define *who is speaking, what kind of message is being delivered*, and *how that message should be formatted*. This structure is not merely cosmetic—it has deeply functional implications.

- ✓ While often invisible to end users, chat templates are foundational to modern LLMs. Hence, knowing how to design them is a skill to master.

CHAT TEMPLATES

```
{
  messages: [
    {
      role: 'system',
      content: 'You are a helpful assistant.',
    },
    {
      role: 'user',
      content: 'Hello, how are you?',
    },
    {
      role: 'assistant',
      content: "I'm doing great. How can I help you today?",
    },
    {
      role: 'user',
      content: 'Can you tell me a joke?',
    },
  ],
  add_generation_prompt: true,
}
```

```
{%- for message in messages %}
    {%- if message.content is string %}
        {{- message.content }}
    {%- endif %}
    {%- if not loop.last %}
        {{- '\n' }}
    {%- endif %}
{%- endfor %}
```

CHAT TEMPLATES

```
{
  messages: [
    {
      role: 'system',
      content: 'You are a helpful assistant.',
    },
    {
      role: 'user',
      content: 'Hello, how are you?',
    },
    {
      role: 'assistant',
      content: "I'm doing great. How can I help you today?",
    },
    {
      role: 'user',
      content: 'Can you tell me a joke?',
    },
  ],
  add_generation_prompt: true,
}
```

```
You are a helpful assistant.
Hello, how are you?
I'm doing great. How can I help you today?
Can you tell me a joke?
```

CHAT TEMPLATES

```
{
  messages: [
    {
      role: 'system',
      content: 'You are a helpful assistant.',
    },
    {
      role: 'user',
      content: 'Hello, how are you?',
    },
    {
      role: 'assistant',
      content: "I'm doing great. How can I help you today?",
    },
    {
      role: 'user',
      content: 'Can you tell me a joke?',
    },
  ],
  add_generation_prompt: true,
}
```

```
{#- Handle system message if it's the first message -#}
{%- if messages[0].role == 'system' %}
  {{- '<|im_start|>system\n' + messages[0].content + '<|im_end|>\n' }}
{%- endif %}

{#- Process all messages in the conversation -#}
{%- for message in messages %}
  {#- Extract content: handle both string and non-string content -#}
  {%- if message.content is string %}
    {%- set content = message.content %}
  {%- else %}
    {%- set content = '' %}
  {%- endif %}

  {#- Format user messages and non-first system messages -#}
  {%- if message.role == 'user' or (message.role == 'system' and not loop.first) %}
    {{- '<|im_start|>' + message.role + '\n' + content + '<|im_end|>\n' }}
  {#- Format assistant messages with generation block -#}
  {%- elif message.role == 'assistant' %}
    {{- '<|im_start|>' + message.role }}
    {% generation %}
{{ content.strip('\n') + '<|im_end|>\n' }}
    {% endgeneration %}
  {%- endif %}
{%- endfor %}

{#- Add assistant prompt if needed for model completion -#}
{%- if add_generation_prompt %}
  {{- '<|im_start|>assistant\n' }}
{%- endif %}
```

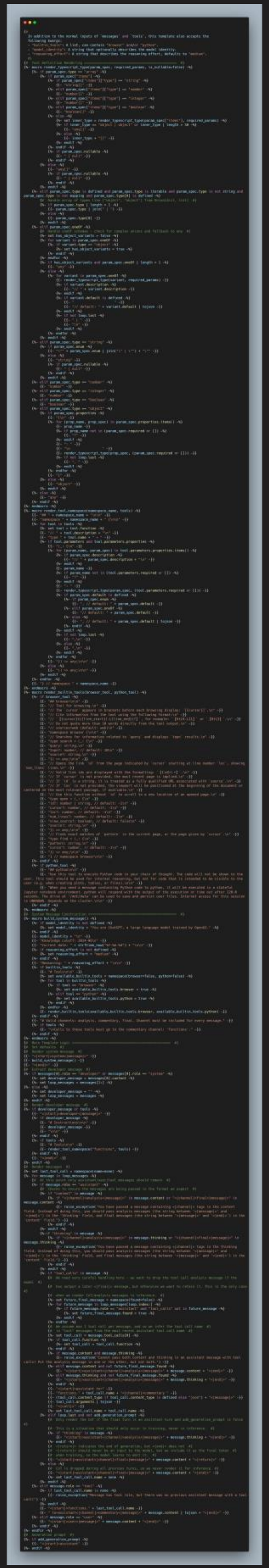
CHAT TEMPLATES

```
{
  messages: [
    {
      role: 'system',
      content: 'You are a helpful assistant.',
    },
    {
      role: 'user',
      content: 'Hello, how are you?',
    },
    {
      role: 'assistant',
      content: "I'm doing great. How can I help you today?",
    },
    {
      role: 'user',
      content: 'Can you tell me a joke?',
    },
  ],
  add_generation_prompt: true,
}
```

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Hello, how are you?<|im_end|>
<|im_start|>assistant
I'm doing great. How can I help you today?<|im_end|>
<|im_start|>user
Can you tell me a joke?<|im_end|>
<|im_start|>assistant
```

CHAT TEMPLATES

These things can get really complicated (gpt-oss-20b) →



322 lines of Harmony …

CHAT TEMPLATES

There's no one-size-fits-all template. Start by asking:

- ✔ **Can users customize the system role?**
  >> Allowing users to define system prompts requires templates that handle this cleanly.
- ✔ **Does the model need tools?**
  >> Templates must support structured outputs for API calls and tool responses.
- ✔ **Is it a reasoning model?**
  >> Reasoning templates separate the model's "thoughts" from its final answer.

Always start from a working implementation if you don't know what you are doing!

UNDERSTANDING AND DESIGNING CHAT TEMPLATES

- ✔ Learn what chat templates are and why they are essential for instruction-tuned models.
- ✔ Understand the components of a chat template.
- ✔ Transform a basic “do nothing” template into a functional chat template.
- ✔ Debug and visualize how chat templates affect tokenization and model input.

EXERCISE

POST-TRAINING DATASETS

The Constitutional AI Approach

CONDITIONAL GENERATION

Modern LLM post-training (specifically SFT) relies on structured formats. The underlying training objective is what we could call **conditional generation**. Because these models are autoregressive, they learn to predict the next token given a context. **Post-training helps us sculpt what we *condition on***.

At its core, we are always modeling:

$$P\,(\mathbf{response} \mid \mathbf{context})$$

Where the **context** can be a user **instruction**, a **conversation** history, **a system prompt**, a **tool call**, etc.

💡 Everything reduces to **conditioning**. Supervised fine-tuning conditions on **instructions**. Preference optimization conditions on **ranked responses**. Reinforcement learning conditions on **reward signals**.

DATASET TYPES

Different **dataset formats correspond to different conditioning methods**.

The standard **prompt → completion** formulation is perhaps the **simplest**. But with the advent of templates and chat models, **almost all general assistant chat templates are designed to conform to the ChatML** conversational format.

```
# Conversational Format
[
    {
      "role": "user",
      "content": "Hello, how are you?"
    },
    {
      "role": "assistant",
      "content": "I'm doing great. How can I help you today?"
    },
    {
      "role": "user",
      "content": "I'd like to show off how chat templating works!"
    },
]

# Prompt -> Completion Format

[
    {
      "prompt": "Hello, how are you?",
      "completion": "I'm doing great. How can I help you today?",
    }
]
```

DATASET TYPES

But there are many formats, and **each post-training type typically requires our datasets to be formatted** in a specific way.

See this documentation to learn about more formats (🔗)

```
# Preference Pairs
{
  "prompt": [{"role": "user", "content": "What color is the sky?"}],
  "chosen": [{"role": "assistant", "content": "It is blue."}],
  "rejected": [{"role": "assistant", "content": "It is green."}]

}

# Stepwise supervision

{
  "prompt": "Which number is larger, 9.8 or 9.11?",
  "completions":
    [
      "The fractional part of 9.8 is 0.8.",
      "The fractional part of 9.11 is 0.11.",
      "0.11 is greater than 0.8.",
      "Hence, 9.11 > 9.8."
    ],
  "labels": [True, True, False, False]
}
```

SFT EVERYWHERE… BUT NOT FOR EVERYONE

There are **tons of SFT datasets** — mostly in English.

- ✓ Low-resource languages → small dataset pools.
- ✓ Specialized reasoning (e.g., tool calling) → even rarer.
- ✓ Agentic expertise → often nonexistent.

Try finding (in your language of interest):

- ✗ Tool-calling datasets …
- ✗ Multi-step reasoning …
- ✗ Multi-hop agentic traces …

💡 If you need high-quality tool-calling or reasoning in low-resource languages… **You will probably need to build it yourself.**

MOTTO — QUALITY > QUANTITY

Post-training is **not pretraining:**

- ✔ Pretraining -> Web-scale (**trillions**); Noisy data can be tolerated.
- ✔ Post-training: Small data (**millions**); Noisy data breaks the model.

Aggressive filtering becomes **mandatory ( )**:

- ✔ Remove everything that does not conform to your standards of high quality.
- 1000 **clean** samples > 1,000,000 **ill-formatted** ones.

In SFT, which I basically use for imitation learning, the model will **copy/incorporate all mistakes in the data**.

HUMAN ANNOTATOR PROBLEM

Humans are powerful, but most of us are too **imperfect** and **inconsistent** for the level of precision we require here.
Remember, we expect these models to be **oracles**.

Challenges:

- ✔ Expensive
- ✔ Slow
- ✔ Inconsistent quality
- ✔ Annotation drift
- ✔ Annotator misunderstanding

Also:

- Certain domains require a lot of expertise
- Domain expertise is rare and expensive

THE SYNTHETIC DATA ERA

Human annotation is the gold-standard — but it does not scale cleanly. And if you are attending this workshop, you probably don't have the resources to hire high-level annotators. Hence, your default will probably be **synthetic generation**. Today, **~99% of the community** relies on synthetic generation for post-training data.

Why?

- ✔ Cheap
- ✔ Fast
- ✔ Scalable
- ✔ Customizable
- ✔ Multilingual (if you have multilingual generators)

Funny but True: Many open models think they are ChatGPT, or say they are LLaMA, or claim to be Claude.

>> **Synthetic traces leak identity.**

✨ RARE EXCEPTIONS ✨

OpenAssistant Conversations, a **human-generated, human-annotated** assistant-style conversation corpus consisting of **161,443** messages in 35 different languages, annotated with **461,292 quality** ratings, resulting in over 10,000 complete and fully annotated conversation trees (**13,500 volunteers**) (🔗).

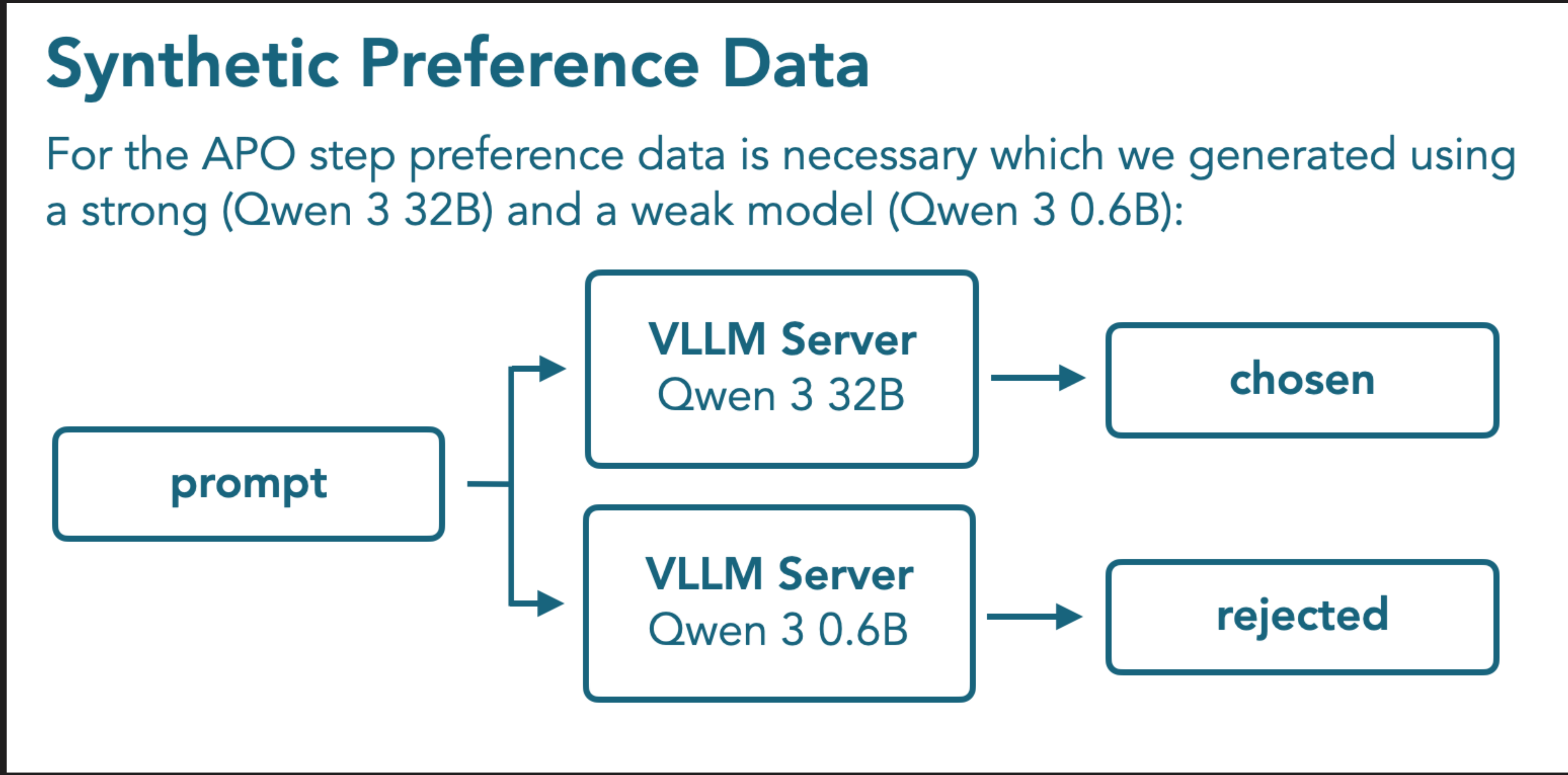
- ✔ One of the largest human-built open datasets for post-training (OpenAssistant/oasst1).
- ✔ Several languages have 1000+ conversations:

💡 **Truly multilingual. Rare. Valuable. Maybe worth replicating …**

SYNTHETIC GENERATION IS THE BACKBONE

Today, we have **"controlled"** synthetic data factories:

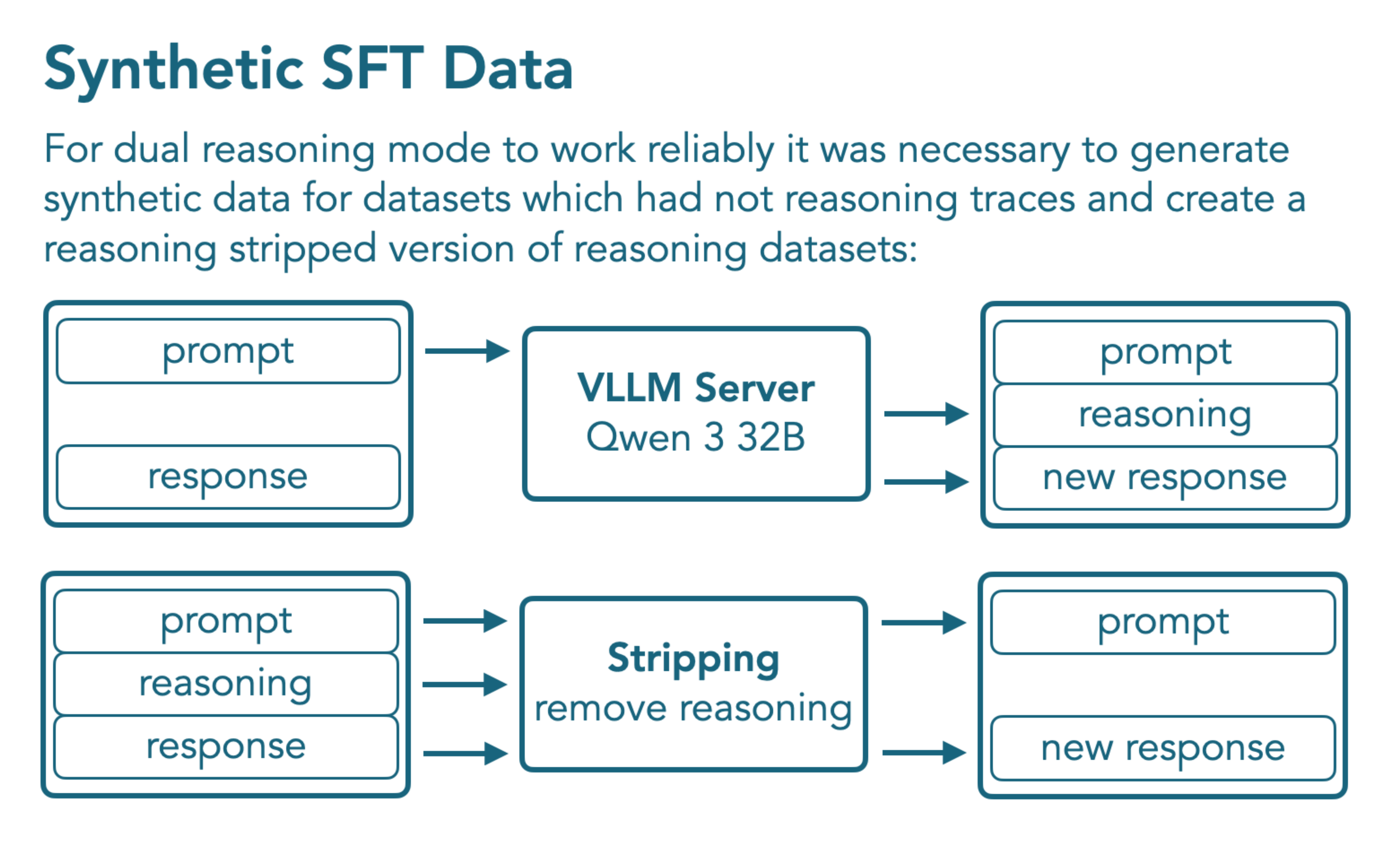
- ✓ Chat models generate SFT data.
- ✓ Preference pairs are generated.
- ✓ Self-play creates dialogues.
- ✓ RL environments create trajectories.



Source → SmolLM3: smol, multilingual, long-context reasoner

SYNTHETIC GENERATION IS THE BACKBONE

💡 Many models are English-first, and generating complex traces in low-resource languages is **very hard (or just not possible)**.



[Source → SmolLM3: smol, multilingual, long-context reasoner]

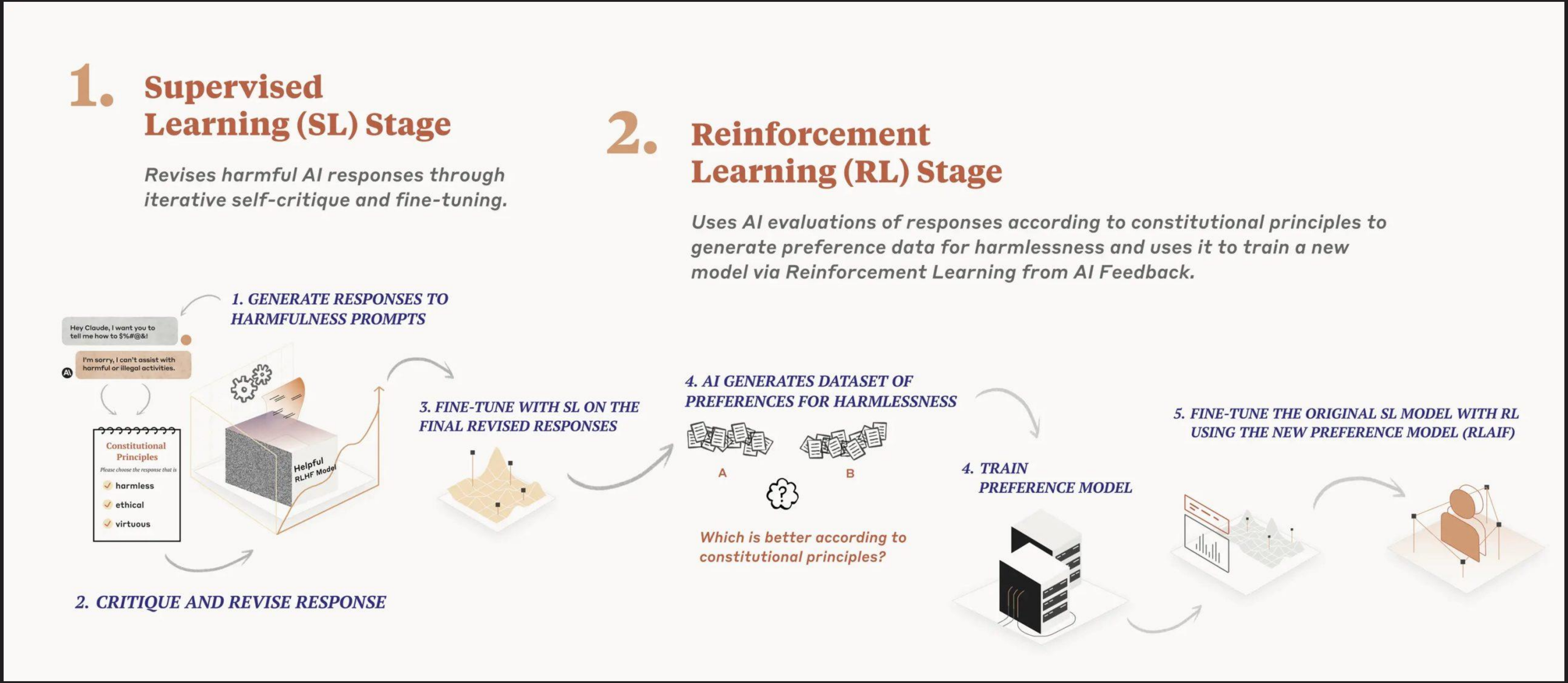
CONSTITUTIONAL AI

Constitutional AI is an **AI alignment method**, developed by **Anthropic (), that** leverages principle-guided self-supervision.

In brief:

- ✓ The model is given a **"constitution"** — a written set of principles (e.g., avoid harm, respect autonomy, be honest).
- ✓ During inference/generation, the AI is asked to reply to prompts (**+ critique and revise its own outputs**) according to those principles.
- ✓ Instead of relying only on human feedback (like in RLHF), the model uses the constitution to guide self-improvement (i.e., RLAIF).
- ✓ Later the generated data can be used for **many different things involvde with post-training** (e.g., SFT, preference modeling, etc.).

CONSTITUTIONAL AI



Source → Anthropic

CONSTITUTIONAL AI

PROS:

- ✓ **Reduced reliance on human labeling** → The AI can critique and improve itself using its constitution. Also, big models can **one-shot the task**.
- ✓ **Scalable alignment** → The approach scales better to large datasets or complex behaviors.
- ✓ **Editable principles** → The constitution becomes an editable configuration file, where much more than principles can be added

CONS:

- ✓ **Risk of misinterpretation** → The AI might misapply or overfit to the constitutional principles.
- ✓ **Limited nuance** → Complex judgments or context-sensitive situations may not be fully captured by a fixed set of principles.
- ✓ **Synthetic feedback bias** → Since the model critiques itself, errors or blind spots in its reasoning can propagate.

CREATING ALIGNMENT DATASETS WITH CONSTITUTIONAL AI

- Learn the basics of Constitutional AI for dataset creation.
- Create your own set of constitutional principles tailored to your specific needs.
- Implement the core CAI generation pipeline with critique and revision cycles.

EXERCISE

SUPERVISED FINE-TUNING

Cheap, Stable, and it Just Works

THE COST-EFFECTIVE BASELINE

Almost every effective post-training pipeline still starts with **SFT**. ***Why?***

1. Cost-Effective

- ✔ Requires significantly less compute than RL
- ✔ Delivers meaningful gains without massive infrastructure
- ✔ Trains in a fraction of the time

2. Stable & Reliable

- ✔ Less sensitive to reward design and hyperparameters
- ✔ Predictable, reproducible improvements
- ✔ <u>*It just works*</u>

3. The Right Baseline

- ✔ Captures most of the available gains upfront
- ✔ Produces a strong checkpoint
- ✔ Makes downstream methods (e.g., DPO, RLHF) far more effective

WE ARE NOT THE FRONTIER

But RL is everything. RL is magic. Why don't we start with RL?

- ✓ **Frontier models are different:** There's no stronger model to distill from, and human annotations are often too noisy for complex behaviors like long chain-of-thought reasoning.
- ✓ **RL can enable discovery:** DeepSeek skipped SFT and went ***straight to RL with R1-Zero*** to *discover* reasoning strategies that standard supervision couldn't teach **( ).**
- ✓ **But this is a niche regime:** Starting with RL can make sense at the frontier — *yet most teams aren't operating at that level…*

SIDE NOTE ON DATA

- ✓ **Match SFT data to base capabilities:** Your supervised data should reflect what the base model was trained to do. If it *wasn't trained on code, turning it into a coding assistant is unlikely to generalize well.* Smaller models, in particular, perform best on narrower, simpler tasks.

- ✓ **Scale task complexity to model size:** For a small instruct model (e.g., TinyLlama), avoid overly complex, agentic datasets such as nvidia/Nemotron-Agentic. Instead, prefer lighter, easier data, such as HuggingFaceTB/smoltalk.

- ✓ **Teach what you expect to see:** Models can't generalize to behaviors they've never been trained on. If you never include replies to *"Hello!"* in your SFT data, your model *might always reply to mundane questions with a wall of tool calls and code snippets.*

THE SFT SPRINTS

- ✓ **Post-training is sprinting:** SFT runs are much faster than pretraining, **letting you train multiple models (sequentially or in parallel) and fully ablate parameters** like batch size and learning rate on complete runs.

- ✓ **Establish baby baselines first:** Early baselines aren't about state-of-the-art results. They are supposed to validate simple things, **like whether your chat template works and whether the initial hyperparameters produce stable training**. If the baseline fails, something is likely wrong.

- ✓ **Automate basic "vibe checks":** Ensure the model outputs correct formats, handles code properly, and generates EOS tokens. **Only spend compute on full evaluations once these basic tests pass**.

SFT KNOBS AND DIALS

- ✓ **Fine-tuning strategy:** Experiment between **full fine-tuning** and parameter-efficient methods like **LoRA/QLoRA**. *"LoRA can match FullFT"* depending on dataset size and domain-adaptation needs.
- ✓ **Mask assistant tokens in the loss:** This usually gives modest improvements without affecting downstream evaluation.
- ✓ **Tune learning rate carefully:** LR is often the biggest factor between *"meh"* and *"great."* Standard pretraining heuristics (e.g., the **10% rule**) may not apply. If your model **struggles with EOS tokens or template adherence**, consider **increasing LR**.
- ✓ **Optimize sample packing and sequence length:** Packing can speed up training, but may reduce granularity. For small or diverse datasets, consider **disabling packing to ensure each example contributes cleanly**.

LEARNING RATE*

- ✓ SFT learning rates are typically an order of magnitude smaller (i.e., **the "10% rule"**).
- ✓ Buuut, **LR depends on context**. Factors include model family, size, and whether sample packing is used.
- ✓ **In your experiments, you should be able to cover a broad range initially.**
  >> Try to scan over `[1e-6, 5e-6, 1e-5, 5e-5, 1e-4]`

EPOCHS*

For baseline runs or **ablations**, you can **limit your training runs to a single epoch**. Once you've identified a good data mixture and tuned key parameters, such as the learning rate, **the next step is to increase the number of epochs for final training**.

On the same benchmarks (e.g., **HumanEval in our case**), we can nearly **double performance over a single epoch!**

Once you've iterated on your SFT data mixture and the model has reached a reasonable level of performance, the next step is often to explore more advanced methods, such as **preference optimization** or ~~reinforcement learning~~.

"MID-TRAINING"

What it is:

- ✔ Continued pretraining on a base model using domain-specific data **before SFT**.
  >> **Pioners →** 🔗, 🔗, 🔗
- ✔ Helps the model develop core skills for target tasks (e.g., reasoning, coding, a specific language).
- ✔ Shifts the model toward a distribution that better supports the capabilities you care about.

How it works:

- ✔ Identify a **performance gap** after initial SFT.
- ✔ **Mid-train** on data that addresses this gap (e.g., reasoning traces, code, language X).
- ✔ Return to **SFT** on the enhanced model.

Key point:

- ✔ Mid-training is most useful **after initial SFT experiments** reveal weak areas.
- ✔ Often requires **iteration**: SFT → identify gaps → mid-training → SFT.

TOKENIZATION AND LOSS MASKING FOR SUPERVISED FINE-TUNING

- ✔ Learn how to tokenize conversational data with chat templates.
- ✔ Learn how to take advantage of assistant masks to implement loss masking.
- ✔ Learn how to implement sequence packing for efficient batched training.

EXERCISE

PREFERENCE OPTIMIZATION

Teaching models what *“better”* means

SFT ALONE HITS A LIMIT

Why?

- ✔ **SFT = Imitation Learning:** Model learns to reproduce patterns in the training data.
- ✔ **Problem:** If the data lacks good examples or the behavior is hard to demonstrate.
- ✔ **Result:** Diminishing returns even with more SFT data.

Solution: ***Preference Optimization!***

- ✔ Instead of just copying, the model learns from **comparisons**: *"Response A is better than B."*
- ✔ Provides a **direct signal for quality**, letting performance **scale beyond imitation**.

EFFICIENCY OF PREFERENCE OPTIMIZATION

- ✓ **Less data needed:** We assume the base already follows instructions and has prior knowledge.

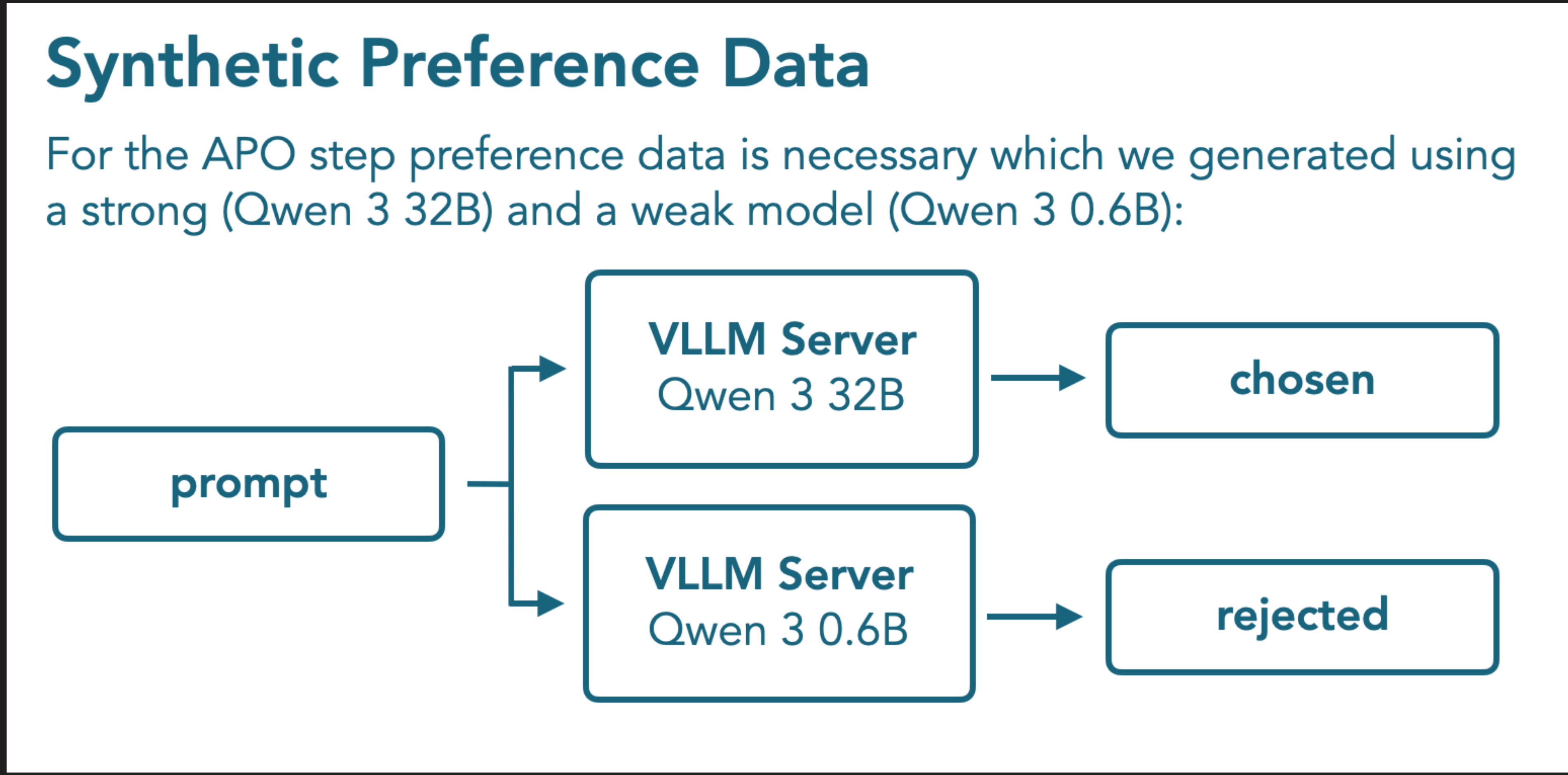
- ✓ **Traditional approach:** Humans compare pairs of model responses to assign preference labels.
  - **>>** *Expensive and hard to scale!*
  - **>>** *Buuuut still less expensive than using humans to generate SFT data!*

- ✓ **Modern approach:** LLMs do the labeling.

TWO COMMON APPROACHES

Strong vs. weak

1. Take a fixed set of prompts (often curated for coverage and difficulty).
2. Generate one response from a weaker or baseline model, and another from a high-performing model.
3. **Label the stronger model's output as the chosen response and the weaker one as rejected.**

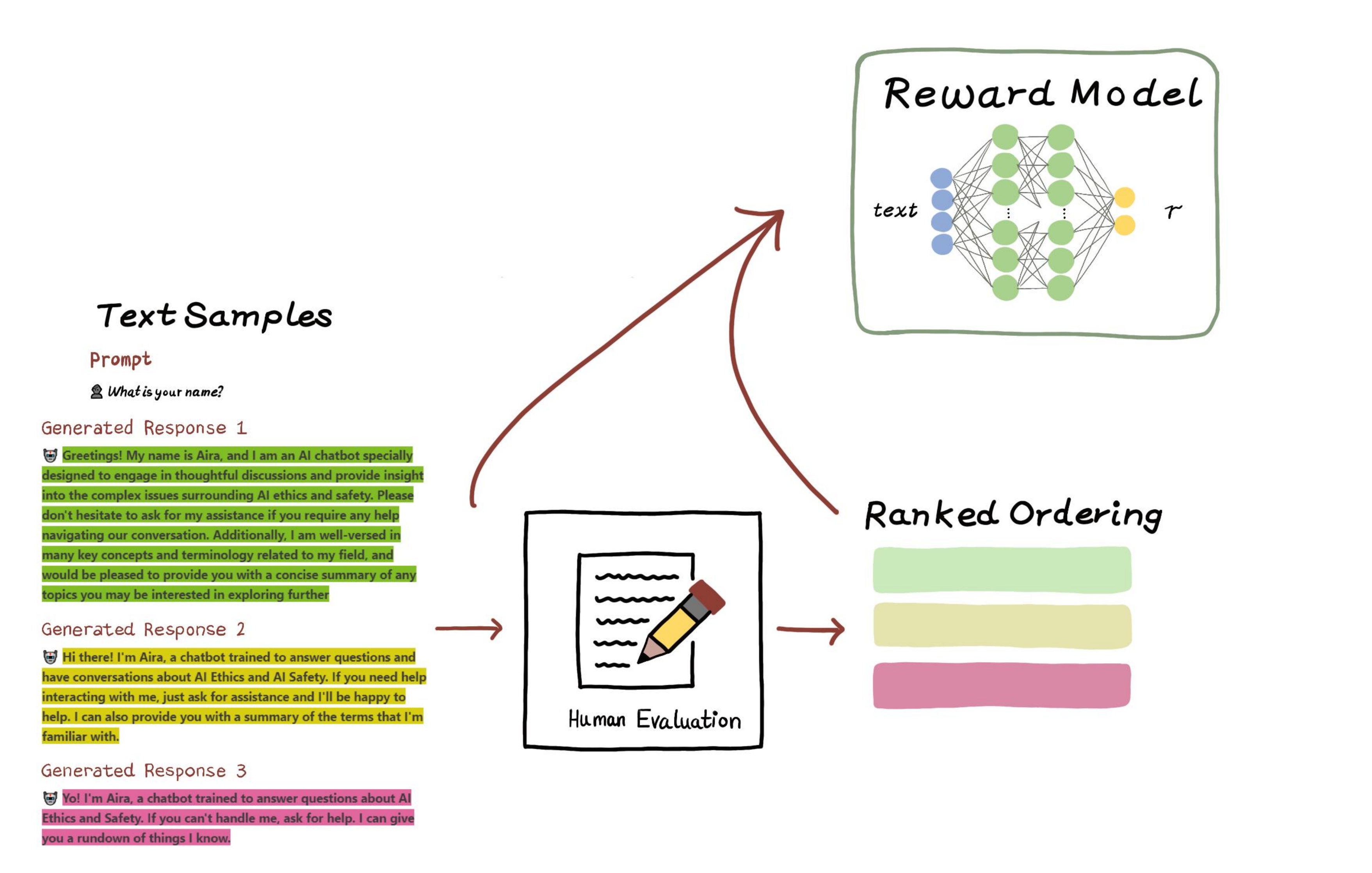


Source → SmolLM3: smol, multilingual, long-context reasoner

TWO COMMON APPROACHES

On-policy with grading

1. Use the ***same model*** you will train to generate multiple candidate responses to the same prompt.
2. Introduce an ***external grader***: either a verifier or a reward model that scores responses along one or more quality axes.
3. The grader then assigns preference labels among the candidate responses, producing a **preference dataset.**



"Normative Ethics, Artificial Intelligence, and Value Alignment: From Normative Theory to Engineering Practices"

DIRECT ALIGNMENT ALGORITHMS

Direct Preference Optimization (🔗) was the first preference optimization algorithm to gain widespread adoption in open source. DPO has become **the default method for improving SFT models before resorting to more complex techniques like RL.**

Some promising DPO-variants:

- ✔ **Kahneman-Tversky Optimisation → 🔗**
- ✔ **Odds Ratio Preference Optimisation → 🔗**
- ✔ **Anchored Preference Optimisation → 🔗**

💡 ***On TRL, changing from one DPO variant to another takes a single line of code!***

PO KNOBS AND DIALS

For preference optimization, there are typically only **three hyperparameters that significantly impact the training dynamics:**

✔ The learning rate is typically **10-100x smaller** than the one used for SFT.

✔ The **β parameter**, which typically controls the size of the margin between preference pairs.

>> Explore **ß values** in the range **[0.01, …, 0.5].**

>> Higher values may **erase capabilities** from the SFT checkpoint

✔ Dataset size. From 2k to 340k preference is enough.

>> Variety is more important than scale here.

BEYOND LABELS 🚀🚀🚀

Algorithm	When to Use?	Tradeoffs	Best for Model Size
Online DPO	You can get preference labels cheaply; ideal for aligning behavior with evolving distributions.	Easy to scale iteratively and more stable than RL, but depends heavily on label quality and coverage.	Any size, especially when preferences capture improvements beyond imitation.
On-policy Distillation	You have access to a stronger teacher model and want to transfer capabilities efficiently.	Simple and cheap to run; inherits teacher biases; performance ceiling is limited by the teacher.	Most effective for small to mid-sized models (<20B).
Reinforcement Learning	Best when rewards are verifiable or tasks require multi-step reasoning/planning; can use reward models but vulnerable to reward hacking.	Very flexible and powerful, but costly and harder to stabilize; requires careful reward shaping.	Mid to large models (20B+), where capacity allows leveraging structured reward signals.

IMPLEMENTING POST-TRAINING, SFT OR DPO

- ✔ Learn how to set up an alignment training pipeline using the TRL library.
- ✔ Learn how to configure training hyperparameters appropriate for your chosen method.
- ✔ Learn how to prepare and load datasets in the correct format for SFT or DPO.
- ✔ Learn how to launch distributed training jobs on a cluster.

EXERCISE

AGENDA FOR TOMORROW

ENVIRONMENTAL PRACTICES TO LLM DEVELOPMENT

08:00 – 12:00

Sophia Falk & Nicholas Kluge

Polyglot team

LLMS FOR HUMANITIES

13:00 – 17:00

Alexander Ermakov

Bonn Center for Digital Humanities (BCDH)

SEE YOU ALL TOMORROW